

Especificacion Proyecto de Simulacion

Llorente Miani

Controlador Simplificado de Bomba de Infusión

21 de junio de 2026

1. Descripción del Sistema

El presente documento detalla la especificación formal en el lenguaje conceptual CML-DEVS de un controlador simplificado para una bomba de infusión. El sistema recibe órdenes médicas, monitorea el caudal administrado mediante un sensor y reacciona ante anomalías temporales y de suministro.

2. Dinámica de Transiciones del Controlador

El componente central del modelo es el **Controlador de la Bomba**, cuya lógica se basa en el manejo de múltiples fases. A continuación se explican sus transiciones principales:

- **Inicio y Detención:** Al recibir el evento `ordenMedica` (δ_{ext}), el sistema transita a la fase `PROCESANDO_ORDEN`. Inmediatamente después (δ_{int}), si el caudal indicado es mayor a cero, pasa a `INFUNDIENDO`; si es cero, asume que debe detenerse y pasa a `ESPERANDO`.
- **Detección y Corrección de Desvíos:** Mientras está `INFUNDIENDO`, evalúa cada lectura del `sensorFlujo`. Si detecta una diferencia mayor al 10% respecto al objetivo, transita a `EVALUANDO_DESVIO`. En esta fase inicia una ventana de tiempo de 5 segundos. Si en ese lapso el caudal vuelve a la normalidad, retorna a `INFUNDIENDO`.
- **Escalamiento a Alarma Crítica:** Si el desvío persiste superado el tiempo de tolerancia, el sistema transita hacia `EVALUANDO_CRITICA` y, de manera automática, a `DETENIDO_POR_CRITICA`. Permanecerá bloqueado en este estado hasta recibir el evento externo `confirmacionEnfermero`.
- **Gestión de Fin de Bolsa:** Al ingresar el evento `finBolsa`, el estado de la bolsa se marca como `FINALIZANDO` y el temporizador σ_{bolsa} se ajusta para vencer en 60 segundos. Si transcurre ese tiempo (se consumen los 60s en las δ_{int}), el sistema detiene la infusión y pasa a `ESPERANDO` con la bolsa `VACIA`.

3. Especificación Formal CML-DEVS

3.1. Generador de Órdenes Médicas

Este componente modela la llegada de prescripciones médicas al sistema. Su función es generar de forma asincrónica eventos ordenMedica que establecen el caudal objetivo que debe administrar la bomba (comprendido entre 0 y 200 ml/h). Las órdenes pueden programarse de manera determinista o estocástica según los escenarios de prueba evaluados.

```
atomic GeneradorOrdenesMedicas is < Y, S,  $\delta_{int}$   $\lambda$ , ta > where

Y is
    out: {Caudal} X R;
end Y

S is
    s: caudal_Indicado X sigma
    caudal_Indicado: R;
    sigma: Time;
end S

 $\delta_{int}((\text{caudal\_Indicado}, \text{sigma}))$  is
    c = U(1-300) // Caudal segun distribucion uniforme
    defcases
        case s = (c, 6000); if (c <= 200) //decision: nuevo caudal cada
            100 minutos
        case s = (0, 6000); if (otherwise)
    end defcases
end  $\delta_{int}$ 

 $\lambda((\text{caudal\_Indicado}, \text{sigma}))$  is
    defcases
        case out = (Caudal, caudal_Indicado); if (true)
    end defcases
end  $\lambda$ 

ta((caudal_Indicado, sigma)) is
    sigma
end ta
end atomic
```

3.2. Sensor de Flujo

Representa el dispositivo encargado de medir y retroalimentar al lazo de control del sistema. Su comportamiento consiste en muestrear periódicamente cada 1 segundo el caudal físico actual neto de la infusión, emitiendo el valor medido a través del puerto sensorFlujo para verificar que no existan desviaciones peligrosas.

```
atomic SensorDeFlujo is < X, Y, S,  $\delta_{int}$   $\delta_{ext}$   $\lambda$ , ta > where

X is
    in: {caudalActual} X R;
end X

Y is
```

```

        out: {sensorFlujo} X R;
    end Y

S is
    s: caudal_medido X sigma
    caudal_medido: R;
    sigma: Time;
end S

 $\delta_{int}$ ((caudal_medido, sigma)) is
    defcases
        case s = (caudal_medido, 1); if (true) //medicion cada 1 segundo
    end defcases
end  $\delta_{int}$ 

 $\delta_{ext}$ ((caudal_medido, sigma), e, (port, value)) is
    defcases
        case s = (value, sigma - e); if (true)
    end defcases
end  $\delta_{ext}$ 

 $\lambda$ ((caudal_medido, sigma)) is
    defcases
        case out = (sensorFlujo, caudal_medido); if (true)
    end defcases
end  $\lambda$ 

ta((caudal_medido, sigma)) is
    sigma
end ta
end atomic

```

3.3. Actuador de la Bomba

Modela el mecanismo físico-mecánico que administra físicamente la medicación intravenosa al paciente. Recibe las señales del controlador central para modificar el caudal real o detenerse por completo. Su comportamiento incluye una restricción temporal de hardware expresada en una latencia máxima de 0.5 segundos para aplicar de forma efectiva cualquier cambio físico de flujo.

```

atomic ActuadorBomba is < X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta > where

X is
    in: ajustarCaudal X R;
    in: detenerBomba X {detener};
end X

Y is
    out: caudalActual X R;
end Y

S is
    s: caudal_objetivo X sigma
    caudal_objetivo: R;
    sigma: Time;
end S

```

```

 $\delta_{int}((caudal\_objetivo, \sigma))$  is
    defcases
        case s = (caudal_objetivo,  $\infty$ ); if (true)
        end defcases
    end  $\delta_{int}$ 

 $\delta_{ext}((caudal\_objetivo, \sigma), e, (port, value))$  is
    defcases
        case s = (value, 0.5); if (port = ajustarCaudal) //0.5 segundos
            de latencia
        case s = (0, 0.5); if (port = detenerBomba) //caudal = 0
            representa bomba detenida
        end defcases
    end  $\delta_{ext}$ 

 $\lambda((caudal\_objetivo, \sigma))$  is
    defcases
        case out = (caudalActual, caudal_objetivo); if (true)
        end defcases
    end  $\lambda$ 

 $ta((caudal\_objetivo, \sigma))$  is
    sigma
end ta
end atomic

```

3.4. Controlador de la Bomba

Es el componente inteligente central del sistema. Se encarga de coordinar todas las entradas (órdenes médicas, lecturas del sensor, fin de bolsa y confirmación del personal de enfermería). Toma las decisiones operativas clave: iniciar la infusión, calcular márgenes dinámicos de desvío de caudal, detener de forma automática la bomba ante emergencias de seguridad y disparar los niveles correspondientes de alarma jerárquica.

```

atomic ControladorBomba is < X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta > where

X is
    in: ordenMedica X R;
    in: sensorFlujo X R;
    in: finBolsa X {fin};
    in: confirmacionEnfermero X {confirmacion};
end X

Y is
    out: ajustarCaudal X R;
    out: detenerBomba X {detener};
    out: Alarma X nivel;
    out: RegistrarEvento X String;
    out: desvio_corregido X {true};
    nivel: {baja, media, critica}
end Y

S is
    s: fase X estadoBolsa X caudal_indicado X caudal_real X sigmaBolsa X
        sigma2

```

```

fase: {
    ESPERANDO, PROCESANDO_ORDEN, INFUNDIENDO, DESVIADO,
    EVALUANDO_DESVIO,
    EVALUANDO_CRITICA, DETENIDO_POR_CRITICA, DESVIO_CORREGIDO
};
estadoBolsa: {ACEPTABLE, FINALIZANDO, VACIA}
caudal_indicado: R;
caudal_real: R;
sigmaBolsa: Time;
sigma2: Time;
end S

 $\delta_{int}$ ((fase, estadoBolsa, caudal_indicado, caudal_real, sigmaBolsa, sigma2))
is
defcases
    // --- PRIORIDAD 1: LA BOLSA ---
    case s = (fase, FINALIZANDO, caudal_indicado, caudal_real, 60,
        sigma2); if (sigmaBolsa = min(sigma2, sigmaBolsa) and esta
        doBolsa = ACEPTABLE)
    case s = (ESPERANDO, VACIA, caudal_indicado, caudal_real,  $\infty$ ,  $\infty$ );
        if (sigmaBolsa = min(sigma2, sigmaBolsa) and estadoBolsa =
        FINALIZANDO)

    // --- PRIORIDAD 2: LAS FASES ---
    case s = (INFUNDIENDO, estadoBolsa, caudal_indicado, caudal_real,
        sigmaBolsa,  $\infty$ ); if (sigma2 = min(sigma2, sigmaBolsa) and
        fase = PROCESANDO_ORDEN and caudal_indicado > 0)
    case s = (ESPERANDO, estadoBolsa, caudal_indicado, caudal_real,
        sigmaBolsa,  $\infty$ ); if (sigma2 = min(sigma2, sigmaBolsa) and fase
        = PROCESANDO_ORDEN and caudal_indicado = 0)
    case s = (EVALUANDO_DESVIO, estadoBolsa, caudal_indicado,
        caudal_real, sigmaBolsa, 0); if (sigma2 = min(sigma2,
        sigmaBolsa) and fase = DESVIADO)
    case s = (EVALUANDO_CRITICA, estadoBolsa, caudal_indicado,
        caudal_real, sigmaBolsa,  $\max(0, 5 - 5*(\text{abs}(\text{caudal\_indicado} -
        \text{caudal\_real})/\text{caudal\_indicado}))$ ); if (sigma2 = min(sigma2,
        sigmaBolsa) and fase = EVALUANDO_DESVIO)
    case s = (DETENIDO_POR_CRITICA, estadoBolsa, caudal_indicado,
        caudal_real, sigmaBolsa, 0); if (sigma2 = min(sigma2,
        sigmaBolsa) and fase = EVALUANDO_CRITICA)
    case s = (INFUNDIENDO, estadoBolsa, caudal_indicado, caudal_real,
        sigmaBolsa,  $\infty$ ); if (sigma2 = min(sigma2, sigmaBolsa) and
        fase = DESVIO_CORREGIDO)
    case s = (fase, estadoBolsa, caudal_indicado, caudal_real,
        sigmaBolsa,  $\infty$ ); if (sigma2 = min(sigma2, sigmaBolsa) and (
        fase = DETENIDO_POR_CRITICA or fase = INFUNDIENDO or fase =
        ESPERANDO))

end defcases
end  $\delta_{int}$ 

 $\delta_{ext}$ ((fase, estadoBolsa, caudal_indicado, caudal_real, sigmaBolsa, sigma2),
e, (port, value)) is
defcases
    case s = (PROCESANDO_ORDEN, estadoBolsa, value, caudal_real,
        sigmaBolsa-e, U(0-3)); if (port = ordenMedica and fase !=
        DETENIDO_POR_CRITICA and estadoBolsa != VACIA)
    case s = (DESVIADO, estadoBolsa, caudal_indicado, value,
        sigmaBolsa-e, 5); if (port = sensorFlujo and  $\text{abs}(\text{value} -$ 

```

```

        caudal_indicado) > (0.1 * caudal_indicado) and fase =
        INFUNDIENDO and estadoBolsa != VACIA)

// El cambio de sigma2 a 0.0 para generar el evento instantaneo de
// desvio corregido
case s = (DESVIO_CORREGIDO, estadoBolsa, caudal_indicado, value,
        sigmaBolsa-e, 0); if (port = sensorFlujo and abs(value -
        caudal_indicado) <= (0.1 * caudal_indicado) and (fase =
        EVALUANDO_CRITICA or fase = EVALUANDO_DESVIO or fase =
        DESVIADO) and estadoBolsa != VACIA)

case s = (fase, estadoBolsa, caudal_indicado, value, sigmaBolsa-e
        , sigma2-e); if (port = sensorFlujo) // Almacena el caudal
        real sin alterar fase
case s = (fase, estadoBolsa, caudal_indicado, caudal_real, 0,
        sigma2-e); if (port = finBolsa)
case s = (PROCESANDO_ORDEN, ACEPTABLE, caudal_indicado,
        caudal_real, ∞, 0); if (port = confirmacionEnfermero)
case s = (fase, estadoBolsa, caudal_indicado, caudal_real,
        sigmaBolsa-e, sigma2-e); if (otherwise)
end defcases
end δext

λ((fase, estadoBolsa, caudal_indicado, caudal_real, sigmaBolsa, sigma2))
is
defcases
// --- PRIORIDAD 1: LA BOLSA ---
case out = (detenerBomba, detener); if (sigmaBolsa = min(sigma2,
        sigmaBolsa) and estadoBolsa = FINALIZANDO)
case out = (Alarma, baja); if (sigmaBolsa = min(sigma2,
        sigmaBolsa) and estadoBolsa = ACEPTABLE)

// --- PRIORIDAD 2: LAS FASES ---
case out = (ajustarCaudal, caudal_indicado); if (sigma2 = min(
        sigma2, sigmaBolsa) and fase = PROCESANDO_ORDEN and
        caudal_indicado > 0)
case out = (detenerBomba, detener); if (sigma2 = min(sigma2,
        sigmaBolsa) and ((fase = PROCESANDO_ORDEN and caudal_indicado
        = 0) or fase = EVALUANDO_CRITICA))
case out = (Alarma, media); if (sigma2 = min(sigma2, sigmaBolsa)
        and fase = DESVIADO)
case out = (ajustarCaudal, caudal_indicado); if (sigma2 = min(
        sigma2, sigmaBolsa) and fase = EVALUANDO_DESVIO)
case out = (Alarma, critica); if (sigma2 = min(sigma2,
        sigmaBolsa) and fase = DETENIDO_POR_CRITICA)
case out = (desvio_corregido, true); if (sigma2 = min(sigma2,
        sigmaBolsa) and fase = DESVIO_CORREGIDO)
end defcases
end λ

ta((fase, estadoBolsa, caudal_indicado, caudal_real, sigmaBolsa, sigma2))
is
min(sigma2, sigmaBolsa)
end ta
end atomic

```

3.5. Módulo de Alarmas

Gestiona la interfaz de notificaciones e interactúa con el personal clínico. Recibe el nivel de alarma gatillado por el controlador principal y procesa su salida sonora/visual. Cuenta con lógica interna de control de tiempos para repetir las alarmas críticas no confirmadas cada 10 segundos, una vez superados los 30 segundos iniciales de tolerancia sin reconocimiento.

```
atomic ModuloAlarmas is < X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta > where

X is
  in: Alarma X nivel;
  nivel: {baja, media, critica}
  in: confirmacionEnfermero X {confirmacion};
end X

Y is
  out: notificacionAlarma X {BAJA, MEDIA, CRITICA};
end Y

S is
  s: fase X tipo X sigma
  fase: {REPOSO, NOTIFICAR_NUEVA, ESPERANDO_30, ESPERANDO_10};
  tipo: {NINGUNA, BAJA, MEDIA, CRITICA};
  sigma: Time;
end S

 $\delta_{int}$ ((fase, tipo, sigma)) is
  defcases
    case s = (REPOSO, NINGUNA,  $\infty$ ); if (fase = NOTIFICAR_NUEVA and
      tipo != CRITICA) //alarmas no criticas se emiten una sola vez
    case s = (ESPERANDO_30, CRITICA, 30); if (fase = NOTIFICAR_NUEVA
      and tipo = CRITICA) //alarma critica se repite por primera
      vez 30s luego de ser activada
    case s = (ESPERANDO_10, tipo, 10); if (fase = ESPERANDO_30)
    case s = (ESPERANDO_10, tipo, 10); if (fase = ESPERANDO_10) //
      alarma critica se repite cada 10s si no es cofirmada
    end defcases
  end  $\delta_{int}$ 

 $\delta_{ext}$ ((fase, tipo, sigma), e, (port, value)) is
  defcases
    case s = (NOTIFICAR_NUEVA, value, 0); if (port = Alarma)
    case s = (REPOSO, NINGUNA,  $\infty$ ); if (port = confirmacionEnfermero)
      //estado pasivo tras confirmar alarma critica
    case s = (fase, tipo, sigma - e); if (otherwise)
    end defcases
  end  $\delta_{ext}$ 

 $\lambda$ ((fase, tipo, sigma)) is
  defcases
    case out = (notificacionAlarma, tipo); if (tipo != NINGUNA)
    end defcases
  end  $\lambda$ 

ta((fase, tipo, sigma)) is
  sigma
end ta
```

```
end atomic
```

3.6. Registrador de Eventos

Consiste en un modelo atómico secundario diseñado para interceptar los cambios operativos de la bomba. Su función exclusiva es actuar como una bitácora o registro (logger) asincrónico del sistema, transformando eventos como cambios de caudal o disparos de alarmas en mensajes textuales legibles con fines de trazabilidad y auditoría técnica

```
//Modelo atomico extra para registrar los eventos generados por el
  controlador de la bomba.

atomic RegistradorEventos is < X, Y, S,  $\delta_{int}$ ,  $\delta_{ext}$ ,  $\lambda$ , ta > where

X is
  in: ajustarCaudal X R;
  in: detenerBomba X {detener};
  in: alarma X nivel;
  nivel: {baja, media, critica};
end X

Y is
  out: registro X String;
end Y

S is
  s: mensaje X sigma;
  mensaje: String;
  sigma: Time;
end S

 $\delta_{int}$ ((mensaje, sigma)) is
  defcases
    case s = ("",  $\infty$ ); if (true)
  end defcases
end  $\delta_{int}$ 

 $\delta_{ext}$ ((mensaje, sigma), e, (port, value)) is
  defcases
    case s = ("Ajuste de caudal", 0); if (port = ajustarCaudal)
    case s = ("Detencion de bomba", 0); if (port = detenerBomba)
    case s = ("Alarma baja", 0); if (port = alarma and value = baja)
    case s = ("Alarma media", 0); if (port = alarma and value =
      media)
    case s = ("Alarma critica", 0); if (port = alarma and value =
      critica)
    case s = (mensaje, sigma - e); if (otherwise)
  end defcases
end  $\delta_{ext}$ 

 $\lambda$ ((mensaje, sigma)) is
  defcases
    case out = (registro, mensaje); if (mensaje != "")
  end defcases
end  $\lambda$ 

ta((mensaje, sigma)) is
```



```

sigma
end ta
end atomic

```

3.7. DEVS acoplado

La estructura final del sistema interconectado y los puertos de los modelos atómicos que conforman el diseño se ilustran en la Figura 1.

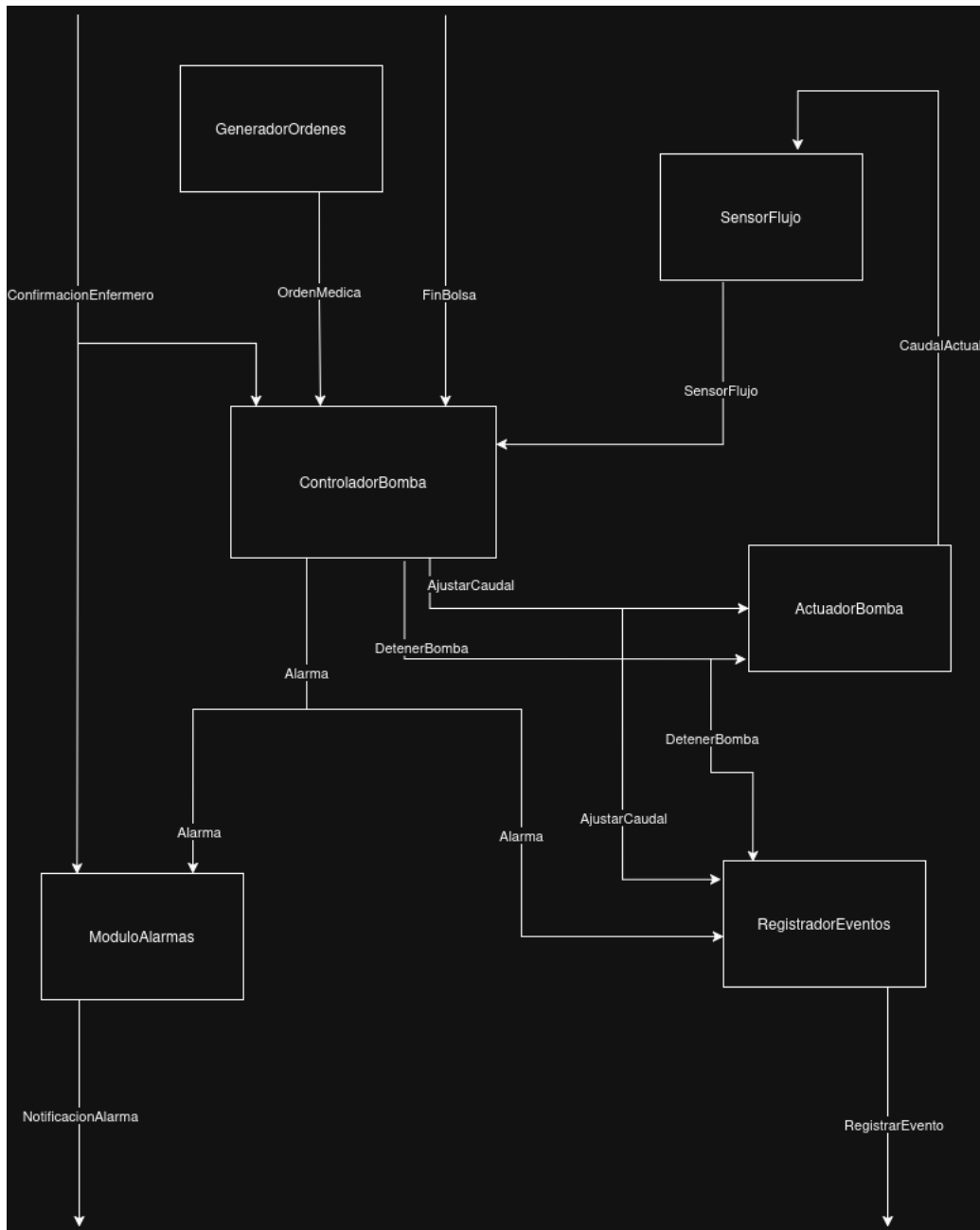


Figura 1: Representación gráfica del modelo DEVS acoplado.

3.8. Nota sobre Estados Iniciales

Nota: No parecía necesario según Hollman pero si se necesita aclarar estos son los estados iniciales de cada DEVS:

- Generador de órdenes médicas: S1 (50, 0)
- Sensor de Flujo: S1 (0, 0)
- Controlador de Bomba: S1 (ESPERANDO, ACEPTABLE, 0, 0, ∞ , ∞)
- Actuador de Bomba: S1 (0, ∞)
- Módulo de Alarmas: (REPOSO, NINGUNA, ∞)
- Registrador de eventos: S1 (, ∞)

4. Descripción de la implementación

La simulación fue implementada en Python utilizando la biblioteca xDEVS, modelando cada componente del sistema de infusión como un modelo atómico DEVS independiente. Se desarrollaron modelos para la bomba, el controlador, el sensor de flujo, la bolsa de infusión, el enfermero y un registrador de eventos, los cuales fueron integrados mediante un modelo acoplado.

Se optó por una arquitectura modular, permitiendo representar de forma separada el comportamiento de cada elemento y facilitar la realización de pruebas sobre distintos escenarios de funcionamiento y falla. Los parámetros del sistema (tiempos de respuesta, tolerancias, duración de la bolsa, errores del actuador, etc.) fueron centralizados en un módulo de configuración, permitiendo modificar fácilmente las condiciones de simulación.

Para el análisis de resultados se utilizó el registrador de eventos para almacenar la evolución temporal de las variables relevantes, las alarmas generadas and las respuestas del sistema. A partir de esta información se obtuvieron métricas como el porcentaje de tiempo con infusión correcta, tiempos de respuesta ante desvíos y reposición de bolsa, además de generar gráficos y trazas cronológicas de la simulación.

5. Resultados de Simulación

5.1. Influencia del error del actuador

Se estudiaron dos escenarios:

- Error aleatorio de $\pm 10\%$
- Error aleatorio de $\pm 15\%$

Analizando para cada caso porcentaje de tiempo con infusión en rango optimo, registro de alarmas generadas, tiempos de respuesta ante desvíos y evolución temporal del caudal de la bomba.

A continuación presentamos ejemplos de resultados obtenidos en cada caso, corriendo una simulación de 10.000 segundos:

- Con error aleatorio de $\pm 10\%$:

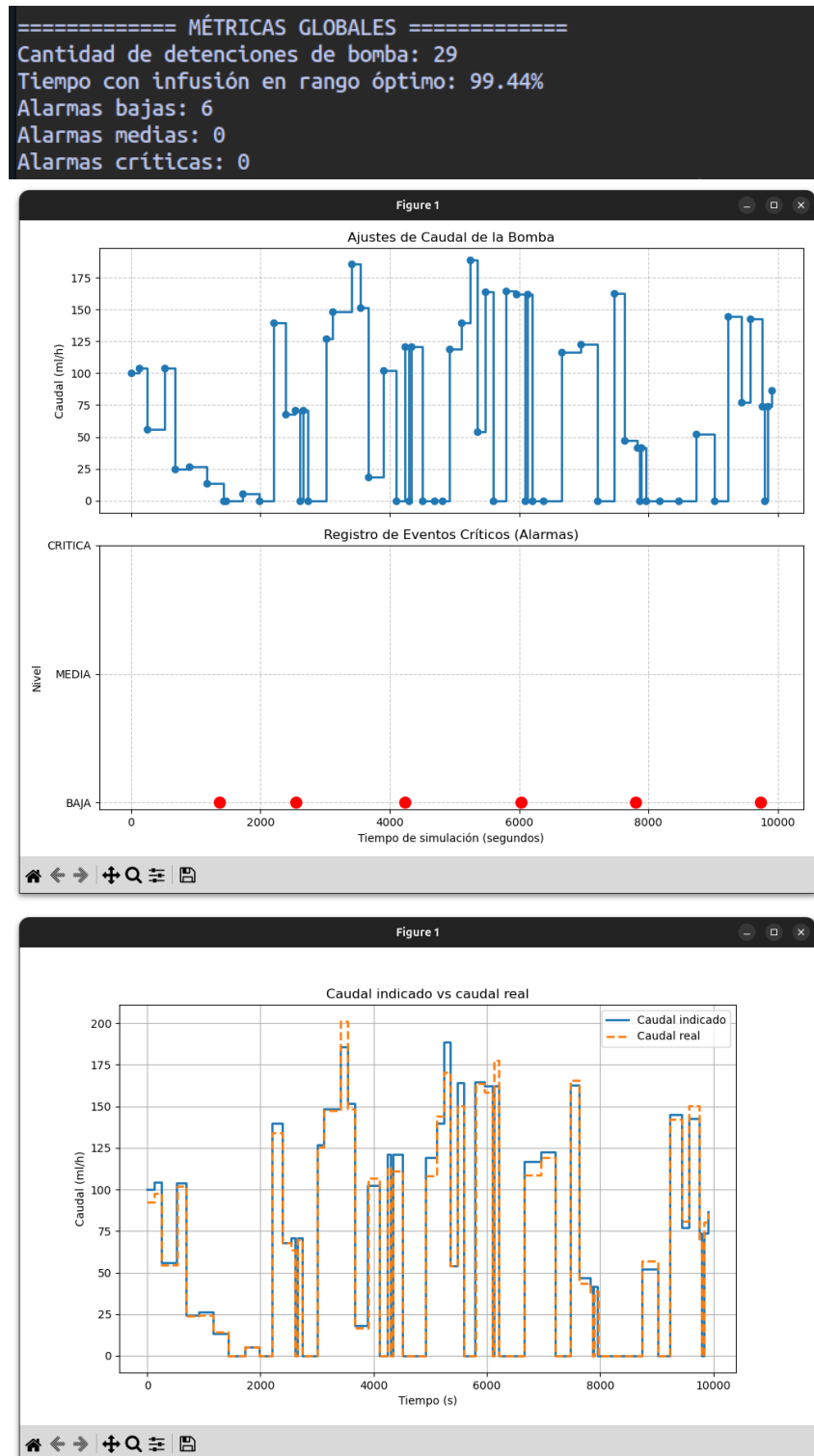


Figura 2: Resultados de simulación con error del actuador al $\pm 10\%$.

Como se observa en la Figura 2, con un error de $\pm 10\%$, no se producen alarmas medias ni críticas, solo alarmas bajas (correspondientes a eventos de fin de bolsa), con un alto porcentaje de tiempo con infusión en rango óptimo (99,44%). Esto se debe a que durante la simulación, aunque se observa cierta diferencia entre el caudal

real y el caudal indicado, esta diferencia no llega a exceder el limite permitido de 10 % por mas de 5 segundos, es por eso que no ocurren alarmas medias ni criticas, a pesar de que la precisión del actuado no es de 100 %.

- Con error aleatorio de $\pm 15\%$:

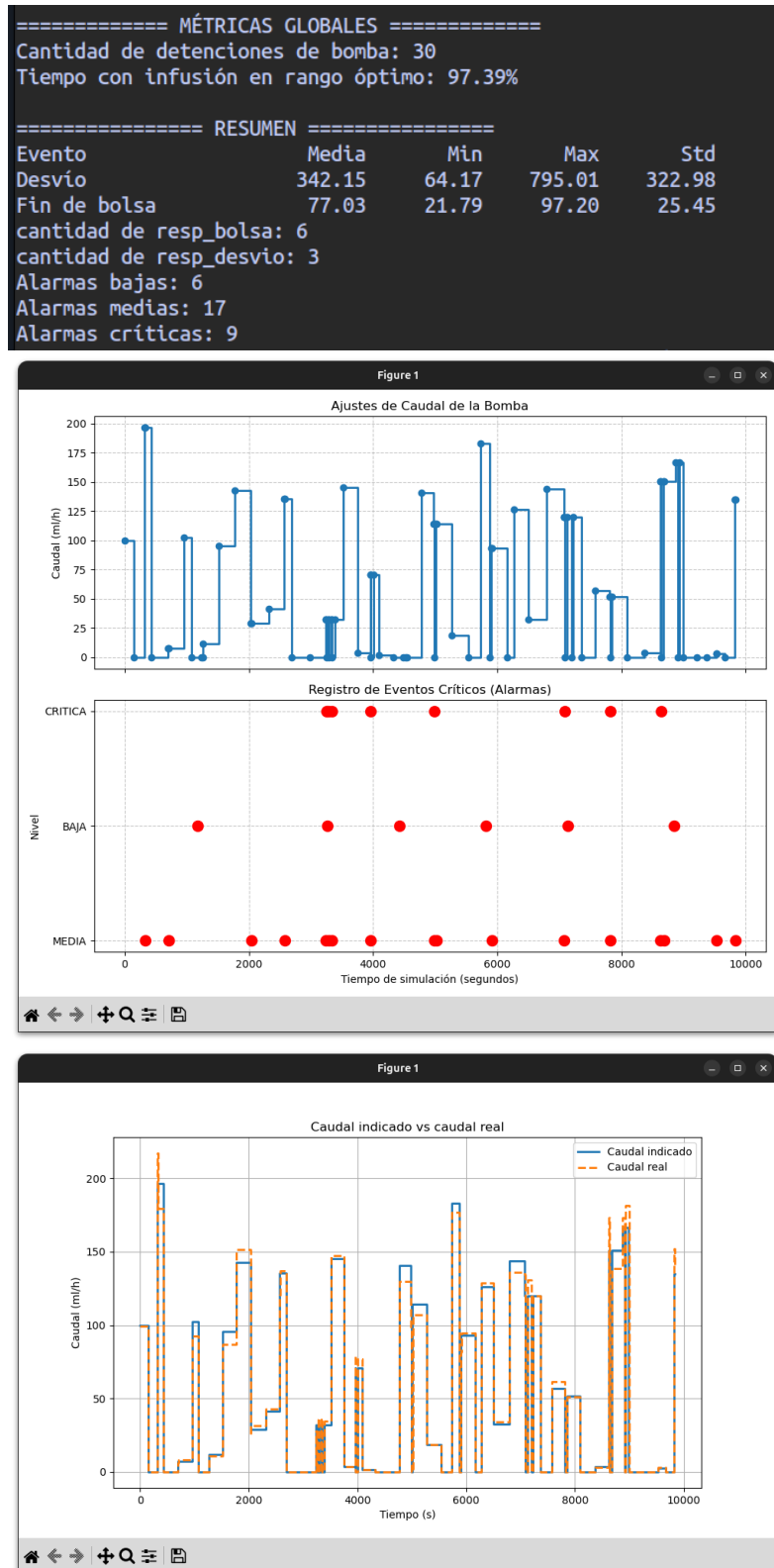


Figura 3: Resultados de simulación con error del actuador al $\pm 15\%$.

Al aumentar el error aleatorio a $\pm 15\%$, como se evidencia en la Figura 3, observamos que el porcentaje de tiempo con infusión en rango óptimo baja considerablemente (97.39 %) y se producen múltiples eventos de alarmas medias y críticas, correspondiente al desvío introducido.

5.2. Influencia del tiempo de respuesta del enfermero

Se ejecutaron escenarios de simulación con tiempos de atención significativamente mayores (entre 1000s y 5000s para alarmas bajas, y entre 500s y 2000s para alarmas críticas). A continuación se presenta un resultado representativo de simulación en la Figura 4.

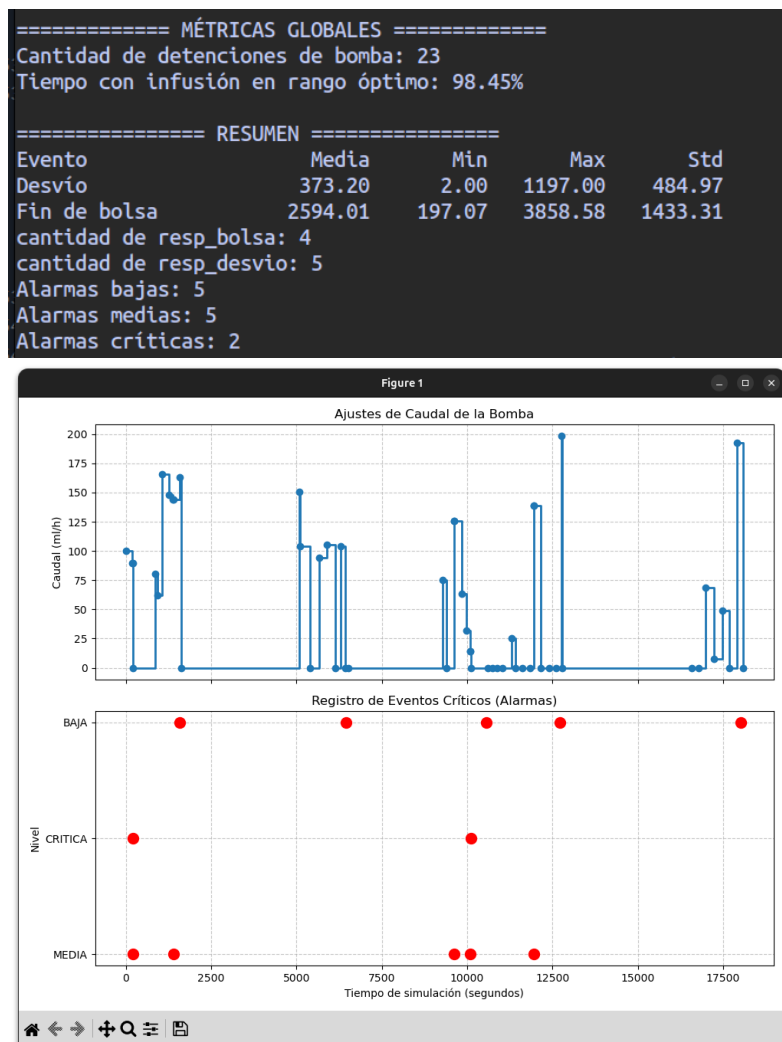
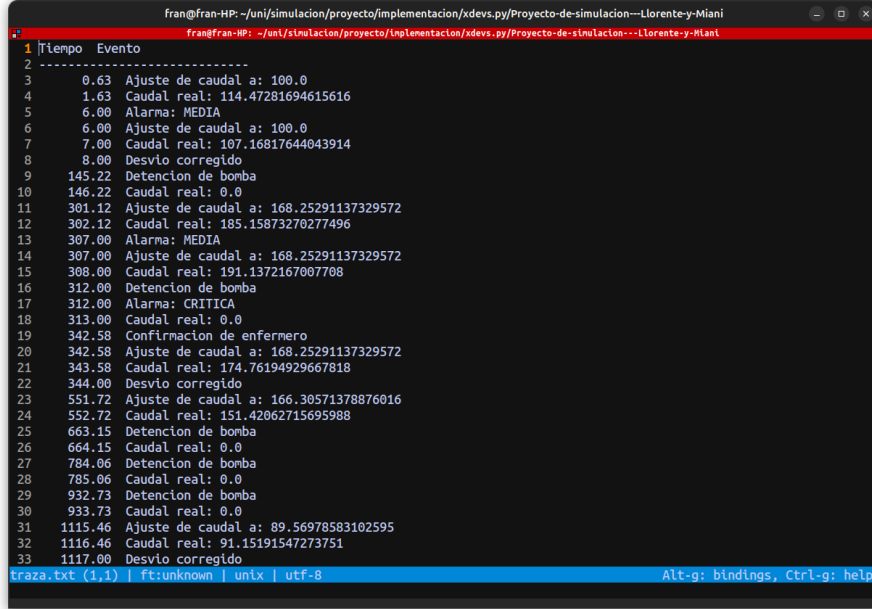


Figura 4: Impacto de la demora en los tiempos de respuesta del personal de enfermería.

Tal como se observa en la Figura 4, con grandes demoras en los tiempos de atención, aumentan drásticamente los tiempos de respuestas del enfermero y esto se ve reflejado en que la bomba de infusión permanece detenida por largos periodos de tiempo.

6. Registro de trazas de simulación

Durante la simulación se almacena una traza cronológica con los eventos relevantes del sistema, permitiendo reconstruir el comportamiento de la bomba y verificar la secuencia de acciones ante situaciones normales y de falla. A continuación presentamos una porción de una traza de simulación obtenida en la Figura 5:



```
fran@fran-HP: ~/uni/simulacion/proyecto/implementacion/xdevs.py/Proyecto-de-simulacion---Llorente-y-Miani
1 |Tiempo  Evento
2 |-----
3 | 0.63  Ajuste de caudal a: 100.0
4 | 1.63  Caudal real: 114.47281694615616
5 | 6.00  Alarma: MEDIA
6 | 6.00  Ajuste de caudal a: 100.0
7 | 7.00  Caudal real: 107.16817644043914
8 | 8.00  Desvto corregido
9 | 145.22 Detencion de bomba
10 | 146.22 Caudal real: 0.0
11 | 301.12 Ajuste de caudal a: 168.25291137329572
12 | 302.12 Caudal real: 185.15873270277496
13 | 307.00 Alarma: MEDIA
14 | 307.00 Ajuste de caudal a: 168.25291137329572
15 | 308.00 Caudal real: 191.1372167007708
16 | 312.00 Detencion de bomba
17 | 312.00 Alarma: CRITICA
18 | 313.00 Caudal real: 0.0
19 | 342.58 Confirmacion de enfermero
20 | 342.58 Ajuste de caudal a: 168.25291137329572
21 | 343.58 Caudal real: 174.76194929667818
22 | 344.00 Desvto corregido
23 | 551.72 Ajuste de caudal a: 166.30571378876016
24 | 552.72 Caudal real: 151.42062715695988
25 | 663.15 Detencion de bomba
26 | 664.15 Caudal real: 0.0
27 | 784.06 Detencion de bomba
28 | 785.06 Caudal real: 0.0
29 | 932.73 Detencion de bomba
30 | 933.73 Caudal real: 0.0
31 | 1115.46 Ajuste de caudal a: 89.56978503102595
32 | 1116.46 Caudal real: 91.15191547273751
33 | 1117.00 Desvto corregido
traza.txt (1,1) | ft:unknown | unix | utf-8 Alt-g: bindings, Ctrl-g: help
```

Figura 5: Fragmento del archivo de traza cronológica de eventos de la simulación.

7. Demostración Formal de Propiedades Temporales

El modelo DEVS diseñado garantiza el cumplimiento de las restricciones temporales críticas del sistema. A continuación se presentan las demostraciones formales basadas en las funciones de transición (δ_{int} , δ_{ext}), avance del tiempo (ta) y salida (λ):

7.1. Inicio de infusión en menos de 3 segundos

Propiedad: Toda `ordenMedica` con caudal mayor que cero debe provocar el inicio de la infusión en menos de 3 segundos.

- Al recibir un evento por el puerto `ordenMedica` con valor $c > 0$, se activa δ_{ext} del Controlador.
- La δ_{ext} actualiza la fase a `PROCESANDO_ORDEN` y programa su próximo evento interno asignando $\sigma \sim U(0, 3)$ (retardo aleatorio acotado).
- Por lo tanto, el tiempo de avance queda acotado: $0 \leq ta(s) \leq 3,0$.
- Al expirar $ta(s)$, el motor invoca a λ , que evalúa `fase = PROCESANDO_ORDEN` y emite el comando `ajustarCaudal`. La acción se dispara en un tiempo menor o igual a 3 segundos.

7.2. Tolerancia de 5 segundos al desvío

Propiedad: Si el caudal difiere más del 10 % respecto del indicado durante más de 5 segundos, debe emitirse una `alarmaMedia`.

- En fase `INFUNDIENDO`, si llega un `sensorFlujo` tal que $|real - indicado| > 0,1 \times indicado$, δ_{ext} cambia a `EVALUANDO_DESVIO` y asigna $\sigma = 5,0$.
- Si no llega un evento que corrija el desvío en ese lapso ($e = 5,0$), ocurre una transición interna.
- Inmediatamente antes de δ_{int} , se ejecuta λ . Al estar en `EVALUANDO_DESVIO`, emite `media` por el puerto `Alarma`, garantizando la emisión exacta al expirar la ventana de gracia.

7.3. Parada máxima a los 60 segundos por fin de bolsa

Propiedad: Si se detecta `finBolsa`, la bomba debe detenerse como máximo luego de 60 segundos.

- Al recibir `finBolsa`, el estado pasa a `FINALIZANDO` y el temporizador paralelo σ_{bolsa} se ajusta a 60.0.
- Frente a eventos externos concurrentes, δ_{ext} descuenta el tiempo transcurrido e ($\sigma'_{bolsa} = \sigma_{bolsa} - e$), impidiendo que otros eventos reinicien este reloj.
- Como $ta(s) = \min(\sigma, \sigma_{bolsa})$, σ_{bolsa} decrece monótonamente hasta 0.
- Al alcanzar 0, λ emite obligatoriamente `detener` por el puerto `detenerBomba` debido a la condición `estadoBolsa = FINALIZANDO`.

7.4. Repetición de Alarma Crítica

Propiedad: Si una `alarmaCritica` no es confirmada dentro de 30 segundos, debe repetirse cada 10 segundos.

- Al recibir `CRITICA`, el `ModuloAlarmas` asigna fase `NOTIFICAR_NUEVA` con $\sigma = 0,0$, emitiendo la alarma inicial.
- Su δ_{int} transita a `ESPERANDO_30` con $\sigma = 30,0$.
- Si no hay confirmación ($e = 30,0$), λ reemite la alarma. La siguiente δ_{int} transita a `ESPERANDO_10` con $\sigma = 10,0$.
- Se forma un ciclo cerrado: el estado `ESPERANDO_10` avanza 10s, invoca λ (reemite) y δ_{int} mantiene el estado con $\sigma = 10,0$, hasta que un evento en `confirmacionEnfermero` fuerza el paso a `REPOSO` con $\sigma = \infty$.

8. Propuestas de Mejora y Trabajo Futuro

Para incrementar la robustez y el realismo del modelo del controlador, se proponen las siguientes mejoras a implementar en iteraciones futuras:

8.1. Estado Inicial de Cebado

El modelo actual asume que el sistema está listo para infundir apenas recibe la primera orden médica. Físicamente, cuando se instala una bolsa nueva, la tubuladura contiene aire que debe ser purgado.

- **Implementación DEVS:** Agregar una fase inicial PURGA previa a ESPERANDO. En este estado, el sistema recibiría un evento manual, ordenando a la bomba expulsar líquido a máxima velocidad durante un tiempo σ_{purga} determinado para extraer el aire, descontando dicho volumen de la bolsa. Una vez finalizada la transición interna de esta fase, el modelo pasaría a ESPERANDO, habilitando el procesamiento de órdenes médicas.

8.2. Silenciado Temporal de Alarmas

Actualmente, cuando el personal de enfermería atiende una alarma (ej. cambiando una bolsa), la señal acústica/visual persiste hasta la confirmación definitiva que reinicia el ciclo, lo cual puede generar fatiga de alarmas y molestias en el entorno clínico.

- **Implementación DEVS:** Extender el `ModuloAlarmas` incorporando un puerto de entrada `i_silenciar`. Al recibir un evento en este puerto, el modelo transitaría mediante δ_{ext} a un estado SILENCIADO con un temporizador acotado (por ejemplo, $\sigma = 120$ segundos), suprimiendo temporalmente la función λ . Si no se recibe una confirmación de resolución del problema subyacente en ese lapso, la δ_{int} devolvería el modelo a su estado de alerta anterior, reanudando la emisión de la alarma.